

**Intelligent Network Intrusion Detection System (NIDS)
Using Deep Learning**

PHASE I REPORT

Submitted by

MYMOON BEEVI B	230701201
SRIVARSHNI S	230701342
RAGHAVAN S	230701520
SRI HARSHAVARDHANAN R	230701337

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

October 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled “**Intelligent Network Intrusion Detection System (NIDS) Using Deep Learning**” is the bonafide work of **MYMOON BEEVI B (230701201), SRIVARSHNI S (230701342), RAGHAVAN S (230701520), SRI HARSHAVARDHANAN R (230701337)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

“This project addresses the following Sustainable Development Goal(s): SDG 9 – Industry, Innovation and Infrastructure ”

SIGNATURE

Dr. J. Manoranjini,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering
College,
Chennai – 602 105.

SIGNATURE

Dr. P. Muneeshwari ,
Associate Professor,
Department of Computer
Science and Engineering,
Rajalakshmi Engineering
College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on

_____ **Internal Examiner External Examiner**

ABSTRACT

Security attacks against networks are constantly rising, which necessitates the development of an intelligent solution that is capable of identifying malicious network traffic. In this project, Intelligent Network Intrusion Detection System (NIDS) using Deep Learning was proposed for multi-class attack detection using the CICIDS2017 dataset. In the proposed approach, the system performs data preprocessing, data cleaning and feature selection using ANOVA and conversion of network flows to sequence for deep learning. In the proposed model, the hybrid CNN-GRU model is used for classifying network traffic in seven different classes: BENIGN, DoS Hulk, DDoS, PortScan, DoS GoldenEye, FTP-Patator, and SSH-Patator. Several different feature sets were considered to see how the feature set affects the performance of the classifier. In this approach, near-perfect accuracy was achieved on the classical stratified test set. However, the Leave-One-Attack-Out (LOAO) evaluation reveals the poor detection capability of the system for the unseen attacks, making the problem of zero day attack detection evident. This project highlights the potentiality of deep learning for classification of known attacks and the necessity of zero-day evaluation and data validation in intrusion detection.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini,, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Dr. P. Muneeshwari, M.E.,Ph.D.**, Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr.S.Madhusudhanan.**, Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

SRIVARSHNI S 230701342

RAGHAVAN S 230701520

SRI HARSHAVARDHANAN R 230701337

MYMOON BEEVI B 230701201

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
	ABSTRACT	
	LIST OF TABLES	
	LIST OF FIGURES	
1.	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 OBJECTIVE	1
	1.3 EXISTING SYSTEM	2
	1.4 PROPOSED SYSTEM	2
2.	LITERATURE REVIEW	3
	2.1 GENERAL	3
	2.2 INTRUSION DETECTION USING DEEP LEARNING	3
	2.3 ZERO-DAY AND UNSEEN ATTACK DETECTION	4
	2.4 FEATURE SELECTION AND REPRESENTATION ANALYSIS	5
	2.5 SUMMARY OF THE REVIEWED WORK	6
	2.6 RESEARCH GAP	7

3.	SYSTEM DESIGN	
7		
	3.1 SYSTEM ARCHITECTURE	9
	3.2 SYSTEM REQUIREMENTS	9
	3.2.1 SOFTWARE REQUIREMENTS	9
	3.2.2 HARDWARE REQUIREMENTS	
11		
	3.2.3 DATASET REQUIREMENTS	11
	3.3 DATASET DESCRIPTION	12
	3.4 DATA PREPROCESSING MODULE	13
	3.5 FEATURE SELECTION MODULE	13
	3.6 SEQUENCE CONSTRUCTION MODULE	
14		
	3.7 CNN-GRU MODEL DESIGN	
14		
	3.8 EVALUATION DESIGN	15
	3.8.1 CLOSED-SET EVALUATION	15
	3.8.2 LEAVE-ONE-ATTACK-OUT (LOAO) EVALUATION	15
	3.8.3 DATA LEAKAGE AUDIT	16
	3.8.4 FEATURE-SPACE ANALYSIS	16
	3.9 EVALUATION METRICS	16
4.	IMPLEMENTATION	17
	4.1 IMPLEMENTATION ENVIRONMENT	17

4.2 DATA CLEANING	17
4.3 FEATURE SELECTION AND SCALING	18
4.4 SEQUENCE GENERATION AND MODEL TRAINING	18
4.5 LOAO IMPLEMENTATION	18
4.6 DATA LEAKAGE AUDIT	19
4.7 FEATURE-SPACE VISUALISATION	19
4.8 INTERACTIVE DEMONSTRATION	19
5. RESULTS AND DISCUSSION	19
5.1 FEATURE SELECTION RESULTS	19
5.2 CLOSED-SET CLASSIFICATION RESULTS	21
5.3 DATA LEAKAGE AUDIT FINDINGS	21
5.4 UNSEEN-ATTACK GENERALIZATION RESULT	21
5.5 FEATURE-SPACE ANALYSIS	24
5.6 DISCUSSION	25
5.7 LIMITATIONS	25
6. CONCLUSION AND FUTURE WORK	26
6.1 CONCLUSION	26
6.2 FUTURE WORK	26
REFERENCES	

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Summary of the reviewed studies	6
3.1	Software Requirements of the proposed system	9
3.2	Hardware Requirements of the proposed system	11
3.3	Traffic classes used in the project	12
3.4	Evaluation metrics	16
5.1	Findings of the data leakage audit (Top - 30 Features)	21
5.2	Qualitative summary of LOAO recall	23

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	Overall methodology of the CNN-GRU based attack-family-level zero-day generalization analysis	15
5.1	Classification accuracy for different feature configurations	20
5.2	Macro F1 - Score for different feature configuration	20
5.3	Recall of the CNN-GRU model on the held-out attack families	22
5.4	F1-score of the CNN-GRU model on the held-out attack families	22
5.5	Comparative LOAO performance across the evaluated attack families	23
5.6	t-SNE visualisation of the Top-30 feature representation across traffic classes	24

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Network security is one of the major concerns in today's computer networks, as huge amounts of network traffic are continually transmitted across the systems. Any malicious act in the network traffic could result in loss of confidentiality, integrity, and availability of the network resources. Network Intrusion Detection System (NIDS) is used to monitor the network traffic and find suspicious activities in the traffic. Due to the increased complexity of the network attacks, intelligent approaches are needed in order to analyze the network traffic and detect different attack types.

Machine Learning and Deep Learning approaches help learn patterns from the network traffic and classify different types of network activities. This project presents Intelligent Network Intrusion Detection System Using Deep Learning using the CICIDS2017 dataset. In the project, seven traffic types have been considered: BENIGN, DoS Hulk, DDoS, PortScan, DoS GoldenEye, FTP-Patator, and SSH-Patator.

1.2 OBJECTIVE

In this project, the objective is to design a deep learning based system which detects and classifies different types of network traffic. The project includes data pre-processing and data quality analysis for training the model. Any invalid values, anomalies, duplicate feature vectors, constant features, and exact duplicate features are excluded from the dataset.

Another goal is to determine significant features of network traffic through

ANOVA F-test and assess different sets of these features. The chosen features are normalized and transformed into sequences of network flows, which are then fed to the CNN-GRU model. Furthermore, this research intends to perform an assessment of the model beyond conventional procedures through LOAO evaluation to detect unseen attack classes and conduct a data leakage audit to assess the reliability of the evaluation procedure.

1.3 EXISTING SYSTEM

Network intrusion detection systems utilize certain procedures like signature, rules, statistical techniques, and machine learning algorithms to detect network activities that could be considered as attacks. Traditional ML and DL models usually separate the given dataset into training and test datasets and assess the model with the help of indicators like accuracy, precision, recall, and F1-score. The mentioned procedures may ensure good results of classification in case attack patterns in the test dataset resemble those seen during training.

Nevertheless, this train/test strategy will not be able to reflect true detection capabilities of the system on previously unknown attacks. It is quite possible that feature patterns will be similar or even identical in both datasets (train and test) leading to overly good results. This project found considerable overlap of the feature vectors of train and test datasets during the leakage analysis.

1.4 PROPOSED SYSTEM

This proposed system will employ hybrid deep learning CNN-GRU architecture for multiclass network intrusion detection problems. First of all, the CICIDS2017 dataset will be cleaned and preprocessed. Afterwards, ANOVA-based feature selection algorithm will be applied to the data. Several different feature sets will be analyzed and the chosen features will be standardized and converted into

sequence tensors consisting of 10 network flows.

The new system tests both familiar attacks and attacks that are not known before. Regular stratified testing technique is applied for the assessment of classification quality on the chosen types of traffic, while LOAO testing leaves out one family of attacks from the training data to test zero-day generalization capability of the system. In addition, another test of data leakage is carried out to discover whether there is any overlap in feature vectors between training and testing data sets.

CHAPTER 2

LITERATURE REVIEW

2.1 GENERAL

A large body of work has been published on learning-based intrusion detection over the last several years. To place this project in context, the studies reviewed here are grouped into three themes: deep learning architectures for intrusion detection, detection of zero-day or unseen attacks, and feature selection with representation analysis. The chapter closes by stating the gap that this project addresses.

2.2 INTRUSION DETECTION USING DEEP LEARNING

Hybrid networks that combine convolutional and recurrent layers are common in the recent literature. Bamber et al. [1] proposed a hybrid CNN-LSTM model for cyber intrusion detection in which convolutional layers pull out local patterns and the recurrent part models the order of the data. Hore et al. [2] followed a sequential deep learning approach aimed at making a network intrusion detection system

more robust and resilient across different kinds of network attacks. Imrana et al. [8] described CNN-GRU-FF, a double-layer feature fusion design built from convolutional and gated recurrent units, which is close in spirit to the CNN-GRU model used in this project.

Comparative work has also been done on the datasets themselves. Arcos-Argudo et al. [3] compared classical machine learning models with hybrid deep representation models on NSL-KDD and CICIDS2017 and reported that the performance of a model can change considerably from one dataset or attack category to another. Chinnasamy et al. [6] surveyed deep learning methods for network-based intrusion detection and observed that CNNs, recurrent networks, autoencoders and hybrid architectures have become the most widely used families of methods.

One point is shared by most of these studies. They are evaluated on a conventional train-test split, so the attack categories seen in testing are also present in training. Results obtained this way show how well a model separates known classes. They say little about how it behaves when an attack family it has never seen appears on the network.

2.3 ZERO-DAY AND UNSEEN ATTACK DETECTION

A practical detection system has to cope with attacks that were absent from its training data. Some researchers have tried to imitate this situation by removing selected attack categories from training and testing the model on them afterwards. Rushendra et al. [4] examined a CNN-LSTM-DQN intrusion detection framework on zero-day and drifted network traffic and found that detection performance can change noticeably when the traffic distribution shifts. The comparison reported by Arcos-Argudo et al. [3] points the same way: models that look strong under the

usual evaluation can lose a great deal of accuracy once an attack category is treated as unseen.

Across these works, the drop in performance is usually reported as a single figure or used to rank competing architectures. What is discussed much less is the variation between attack families. An attack may be easy or hard to detect depending on how closely its traffic resembles the benign traffic and the attack families that were used for training. That question is the focus of this project.

2.4 FEATURE SELECTION AND REPRESENTATION ANALYSIS

Network traffic datasets carry many features, and some of them are redundant or add very little information. Reducing the number of features lowers the computing cost and can help a model concentrate on the more useful signals. Umer et al. [5] combined wrapper-based feature selection with multi-head attention transformers for intrusion detection. Otokwala et al. [7] proposed an optimized common feature selection method together with a deep autoencoder (OCFSDA) for lightweight intrusion detection in Internet of Things settings. In this project the ANOVA F-test is used, which is a simple statistical filter that scores each feature by how well it separates the classes.

Beyond the input features, deep networks build internal representations in their hidden layers. Accuracy, precision, recall and F1-score do not reveal how these representations are arranged. Visualisation methods such as t-SNE [10] map high-dimensional data to two dimensions and can show whether different traffic classes form separate groups or overlap. They are used here strictly as a qualitative aid, and not as a proof that classes can or cannot be separated.

2.5 SUMMARY OF THE REVIEWED WORK

Table 2.1 Summary of the reviewed studies

Ref.	Focus of the study	Relevance to this project
[1]	Hybrid CNN-LSTM for intrusion detection	Supports the use of a hybrid convolutional and recurrent design
[2]	Sequential deep learning for a robust NIDS	Supports treating traffic as ordered sequences of flows
[3]	Classical ML versus hybrid deep models on NSL-KDD and CICIDS2017	Shows that results depend on dataset and attack category, and that unseen categories lower performance
[4]	CNN-LSTM-DQN for zero-day and drifted traffic	Shows that detection changes with traffic distribution
[5]	Wrapper feature selection with multi-head attention transformers	Example of feature selection applied to intrusion detection
[6]	Systematic review of deep learning based NIDS	Gives an overview of CNN, recurrent, autoencoder and hybrid methods
[7]	OCFSDA for lightweight IoT intrusion detection	Example of feature reduction combined with deep learning
[8]	CNN-GRU-FF feature fusion network	Closest to the CNN-GRU architecture used here

2.6 RESEARCH GAP

Three gaps can be identified from the studies above. First, most deep learning based systems are judged only under the conventional split, which can give an over-optimistic picture. Second, studies that do remove attack categories tend to report the overall drop or compare models, and only a few look at why some attack families are harder to recognise than others. Third, a good closed-set score is rarely accompanied by a check of whether the training and test sets share identical records.

This project responds to these gaps by evaluating a CNN-GRU model under a Leave-One-Attack-Out protocol on six attack families of CICIDS2017, by comparing the outcome for each family separately, by inspecting the feature space with t-SNE, and by auditing the data for overlap between the training and test sets.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The architecture of the intelligent network intrusion detection system is sequential. In other words, it converts network traffic data into classified intrusions. The architecture includes multiple stages such as data pre-processing, feature cleaning, splitting of the dataset, feature selection, scaling, sequence generation, CNN-GRU deep learning model training, and evaluation of the model's performance.

The initial stage of the algorithm includes CICIDS2017 raw CSV data that includes both benign network traffic data and various types of attacks. The raw data is pre-processed in order to clean the data from any infinite and missing

values as well as from anomalies and duplicates. Once the dataset was cleaned, the data was stratified to 80% training set and 20% test set. The next step is to apply ANOVA F-test feature selection in order to prioritize features and to create different feature subsets.

The next step is to scale the data using StandardScaler. This algorithm is fitted only on the training data set in order to avoid data leakage. The final step is to create sequences from the scaled data using a window size of 10 network flows. These sequences serve as input data for the CNN-GRU deep learning model.

The final softmax layer produces predictions for the seven network traffic classes.

Following the model training process, the test evaluation is carried out with traditional testing using the stratified test set. Moreover, LOAO is done to assess the ability of the model to generalize when attacked with families not seen during the training stage. A protocol and leakage audit is carried out to detect overlapping feature vectors and label discrepancies between the training and testing sets. Lastly, the trained Top-40 CNN-GRU model can be utilized for the interactive demonstration. The model stresses the significance of ensuring high quality of data before employing the deep learning model. Invalid value, duplicates, constant and redundant features are common in network traffic datasets. Thus, cleaning and feature reduction processes are conducted before the model training process in order to present the most reliable input. It ensures that the CNN-GRU model is supplied with meaningful network traffic features.

The feature selection process helps reduce the dimensionality of the input features via ANOVA F-test. Various subsets of features such as Top-10, Top-20, Top-30, Top-40, and all 61 usable features are tested for the effect of feature

selection on the classification performance. The selected features are then standardized using StandardScaler, with the scaler fitted only on the training data to avoid information leakage from the test set. The data obtained by preprocessing and feature selection is then converted into the sequence of 10 flows. In this way, CNN-GRU will be able to treat the data in a sequential way and not treat each network flow independently. The local patterns of the features sequences are extracted by CNN while the GRU layer learns the sequential patterns of the generated flow windows.

Finally, the last part of the architecture aims at evaluating the robustness of the intrusion detection system. In addition to traditional stratified test, the system is tested in terms of Leave-One-Attack-Out that checks how the learned model behaves with respect to the attack class that is not seen during training. There is also conducted protocol and data leakage audit that finds the overlapping features patterns between training and testing datasets.

3.2 SYSTEM REQUIREMENTS

3.2.1 SOFTWARE REQUIREMENTS

Table 3.1 Software Requirements of the Proposed System

S. No.	Software / Tool	Purpose
1	Python	Primary programming language for preprocessing, feature selection, model training, and evaluation
2	TensorFlow (≥ 2.11)	Development and training of the CNN-GRU deep learning model
3	NumPy	Numerical computations and handling processed datasets

S. No.	Software / Tool	Purpose
1	Python	Primary programming language for preprocessing, feature selection, model training, and evaluation
4	Pandas	Data loading, cleaning, manipulation, and analysis
5	Scikit-learn	ANOVA feature selection, StandardScaler, data splitting, and evaluation
6	Matplotlib	Visualization of experimental results
7	Seaborn	Statistical and graphical visualization
8	CICIDS2017 Dataset	Network traffic dataset containing benign and malicious traffic
9	Windows OS	Development and execution environment
10	Project Scripts and Models	Preprocessing, sequence generation, CNN-GRU training, evaluation, and demonstration

3.2.2 HARDWARE REQUIREMENTS

Table 3.2 Hardware Requirements of the Proposed System

S. No.	Hardware Component	Requirement / Purpose
1	Processor	Multi-core processor suitable for large-scale data processing and model training
2	RAM	Sufficient memory for handling the large CICIDS2017 dataset and intermediate data
3	Storage	Adequate storage for raw/processed datasets, trained models, and results
4	GPU	Not mandatory; the implementation is designed to run in a Windows CPU environment
5	Display	Required for development, result visualization, and system demonstration
6	Keyboard & Mouse	Required for system development and interaction

3.2.3 DATASET REQUIREMENTS

The proposed Intelligent Network Intrusion Detection System uses the CICIDS2017 dataset, which contains benign and malicious network traffic collected over five days. For this project, seven classes are selected: BENIGN, DoS Hulk, DDoS, PortScan, DoS GoldenEye, FTP-Patator, and SSH-Patator. The dataset is cleaned by handling invalid values, removing duplicate and conflicting records, and eliminating redundant features, resulting in 2,805,715 records with

61 usable features for further processing and model development.

3.3 DATASET DESCRIPTION

CICIDS2017 is a labelled network intrusion detection dataset that was built from traffic captured over five days [9]. It holds benign traffic along with several present-day attacks, and this variety makes it suitable for testing how a model behaves on attack families it was not trained on. Seven traffic classes were selected for this work: one benign class and six attack classes. Table 3.3 lists them.

Table 3.3 Traffic classes used in the project

Class	Traffic type	Held out in LOAO
BENIGN	Normal traffic	No
DoS Hulk	Attack	Yes
DDoS	Attack	Yes
PortScan	Attack	Yes
DoS GoldenEye	Attack	Yes
FTP-Patator	Attack	Yes
SSH-Patator	Attack	Yes

After cleaning, the working dataset has 2,805,715 records and 61 usable features, as described in Section 3.2.3. The benign class is always kept in training. Each of the six attack classes is removed from training in turn during the zero-day experiments.

3.4 DATA PREPROCESSING MODULE

The preprocessing module prepares the raw CSV files for learning. It carries out the following steps in order:

- Invalid values such as infinite and missing entries are handled, and anomalous rows are dropped.
- Duplicate and conflicting records are removed, so that no identical feature vector remains attached to two different labels in the cleaned data.
- Constant features and exact duplicate features are eliminated, which leaves 61 usable features.
- The class labels are encoded as integers for multi-class classification.
- The data is divided with stratification into 80% for training and 20% for testing.
- StandardScaler is fitted on the training portion only and is then applied to the test portion, so that no information from the test data influences the scaling.

3.5 FEATURE SELECTION MODULE

The ANOVA F-test gives every feature a score based on the ratio of the variance between classes to the variance within classes. A high score means the feature separates the classes well. The features are ranked by this score and five configurations are prepared: Top-10, Top-20, Top-30, Top-40 and the complete set of 61 features. Each configuration is tested with the same CNN-GRU model so that the effect of the number of features on classification can be observed. The Top-30 configuration is used for the unseen-attack experiments and for the feature-space analysis, since it kept strong classification performance while using fewer than half of the available features.

3.6 SEQUENCE CONSTRUCTION MODULE

Network flows do not occur in isolation, and the flows that arrive one after another often say something about each other. For that reason the scaled feature vectors are grouped into windows of 10 consecutive flows. Every window becomes one input sample of shape 10 by the number of selected features. This lets the recurrent part of the model learn from the order of the flows rather than looking at each flow independently.

3.7 CNN-GRU MODEL DESIGN

The classifier is a hybrid network with three stages. The convolutional stage works along each input sequence and extracts local patterns from the selected flow features. The output of this stage goes to the GRU layer, which captures the temporal relationships among the flows of the window through its gating mechanism. The final stage is a fully connected layer with softmax activation that gives a probability for each traffic class. The same model configuration is kept for every experiment, so that any difference in results comes from the data and the protocol and not from a change in the network.

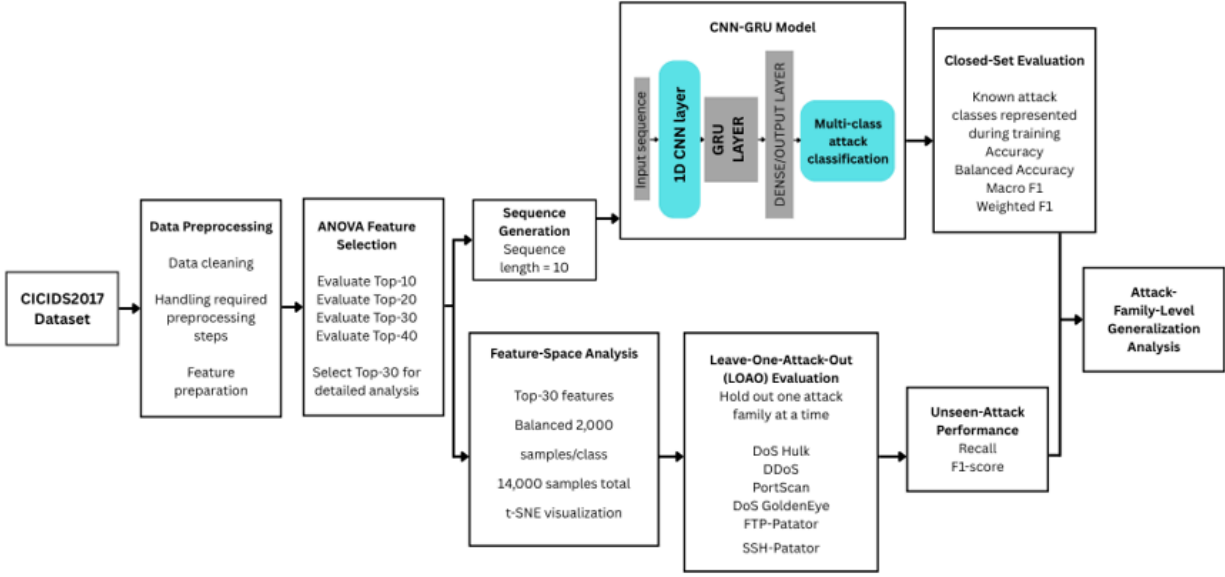


Figure 3.1 Overall methodology of the CNN-GRU based attack-family-level zero-day generalization analysis

3.8 EVALUATION DESIGN

3.8.1 Closed-set evaluation

In the closed-set evaluation the model is trained and tested on the same seven classes using the stratified 80/20 split. Accuracy, balanced accuracy, macro F1-score and weighted F1-score are reported for each feature configuration.

3.8.2 Leave-One-Attack-Out (LOAO) evaluation

To imitate a zero-day situation, one attack family is removed completely from the training data. The model is trained on the remaining classes and is then tested on the removed family, which the model has never encountered. The procedure is repeated for DoS Hulk, DDoS, PortScan, DoS GoldenEye, FTP-Patator and SSH-Patator. Recall and F1-score on the held-out family are the main measures, because recall states directly what share of the unseen attack was caught.

3.8.3 Data leakage audit

The audit compares the training and test feature rows in the Top-30 space and counts the rows that appear in both. It also looks for identical feature signatures that carry different labels. This shows how far the closed-set score can be trusted.

3.8.4 Feature-space analysis

A balanced subset of 2,000 samples per class, 14,000 in total, is drawn in the Top-30 configuration. The subset is projected to two dimensions with t-SNE to look at how the traffic classes are distributed and where they overlap.

3.9 EVALUATION METRICS

The following metrics are used to judge the system, where TP, TN, FP and FN denote true positives, true negatives, false positives and false negatives.

Table 3.4 Evaluation metrics

Metric	Definition	Purpose
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Share of all samples classified correctly
Precision	$TP / (TP + FP)$	Share of raised alarms that were real attacks
Recall	$TP / (TP + FN)$	Share of real attacks that were detected
F1-score	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$	Balance between precision and recall
False positive rate	$FP / (FP + TN)$	Share of benign traffic wrongly flagged

Balanced accuracy	Mean of the recall of each class	Accuracy that is not dominated by large classes
Macro / weighted F1	Class-averaged F1 (equal / size-weighted)	Overall multi-class quality

CHAPTER 4

IMPLEMENTATION

4.1 IMPLEMENTATION ENVIRONMENT

The system was written in Python. Pandas and NumPy handle loading and cleaning of the data, Scikit-learn supplies the ANOVA F-test, StandardScaler, the stratified split and the metrics, and TensorFlow is used to build and train the CNN-GRU network. Matplotlib and Seaborn produce the charts. All experiments were run on a Windows machine using the CPU, since a GPU is not required for this implementation (Section 3.2).

4.2 DATA CLEANING

The raw CICIDS2017 files contain infinite values, missing entries and repeated rows, all of which can disturb training. These were dealt with first. Records that were exact duplicates or that were in conflict with each other were removed, and the columns that never change or that repeat another column were dropped. After these steps 2,805,715 records and 61 features remained for the seven selected classes.

4.3 FEATURE SELECTION AND SCALING

The cleaned data was divided into training and test sets with stratification. The ANOVA F-test was run to rank the features, and subsets of 10, 20, 30, 40 and all 61 features were created. StandardScaler was fitted on the training set alone. The fitted scaler was then used to transform the test set and, later, the held-out attack family in each LOAO run.

4.4 SEQUENCE GENERATION AND MODEL TRAINING

Scaled records were arranged in windows of 10 flows. The CNN-GRU model was trained on these windows for the multi-class task, with the training data kept apart from the data used for evaluation. The training settings were held constant across all the feature configurations and across all LOAO experiments. [Insert here the optimizer, learning rate, batch size, number of epochs and layer sizes used in your notebook.]

4.5 LOAO IMPLEMENTATION

For each of the six attack families the following procedure was repeated. All samples of the chosen family were taken out of the training data. The model was trained from the beginning on the benign class and the five remaining attack classes. After training, the held-out samples were passed through the model and the recall and F1-score for that family were computed, together with accuracy, precision and false positive rate. Because a fresh model is trained for each family, the six results are independent of one another.

4.6 DATA LEAKAGE AUDIT

Feature rows of the training and test sets in the Top-30 space were compared to find the rows that occur in both. Feature signatures that appear with more than one

label were also counted. The purpose of the audit was not to change the results but to decide how they should be read.

4.7 FEATURE-SPACE VISUALISATION

For the visual analysis, 2,000 samples of each of the seven classes were selected at random in the Top-30 configuration, giving a balanced set of 14,000 samples. This set was reduced to two dimensions with t-SNE, and each class was drawn in a different colour.

4.8 INTERACTIVE DEMONSTRATION

The trained Top-40 CNN-GRU model can be used in an interactive demonstration, where network traffic records are fed to the model and the predicted class is displayed. This demonstration is included to show how the trained model would be used, and it does not take part in the LOAO evaluation, which uses the Top-30 setting.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 FEATURE SELECTION RESULTS

The CNN-GRU model was evaluated with the Top-10, Top-20, Top-30, Top-40 and all 61 features. Every reduced configuration kept strong performance under the conventional evaluation, as Figures 5.1 and 5.2 show. Accuracy stays close to 1.000 for all settings, and macro F1-score rises slightly as more features are added and then levels off. Since the smaller feature sets performed almost as well as the larger ones, the Top-30 configuration was chosen for the unseen-attack experiments.

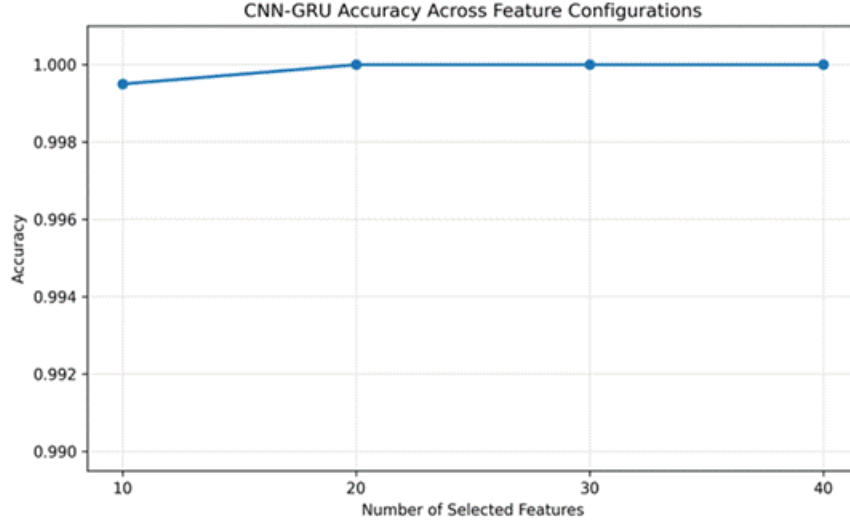


Figure 5.1 Classification accuracy for different feature configurations

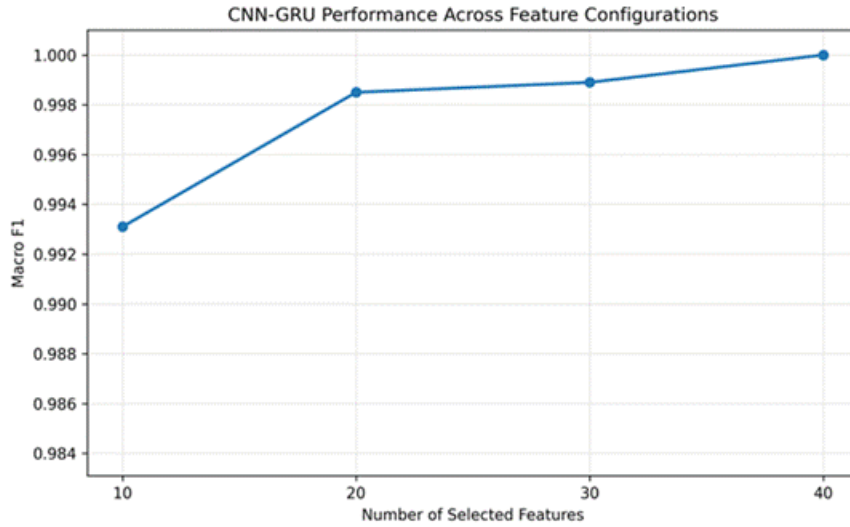


Figure 5.2 Macro F1-score for different feature configurations

5.2 CLOSED-SET CLASSIFICATION RESULTS

With the conventional stratified split, the CNN-GRU model reached accuracy and F1-score approaching 100%. On the face of it, this means the model can classify attack families correctly when those families are part of its training data. The audit

described in the next section, however, shows that these numbers need to be read carefully.

5.3 DATA LEAKAGE AUDIT FINDINGS

The audit found a large overlap between the training and test data in the Top-30 feature space. The findings are given in Table 5.1.

Table 5.1 Findings of the data leakage audit (Top-30 features)

Audit item	Count
Feature rows common to the training and test sets	45,848
Label-inconsistent duplicate signatures	81

A test set that shares rows with the training set makes the task easier, because the model is partly asked to recognise what it has already seen. The label-inconsistent signatures mean that a few identical feature vectors are tied to different classes, which no classifier can separate using those features. For these reasons the near-perfect result of Section 5.2 should be taken as closed-set performance and not as evidence that the model handles genuinely new traffic well.

5.4 UNSEEN-ATTACK GENERALIZATION RESULTS

The LOAO results are very different from the closed-set ones. Over the six held-out attack families, the CNN-GRU model reached a mean recall of 6.23% and a mean F1-score of 11.50%. In other words, on average the model found only about six out of every hundred flows of an attack family that it had not seen during training.

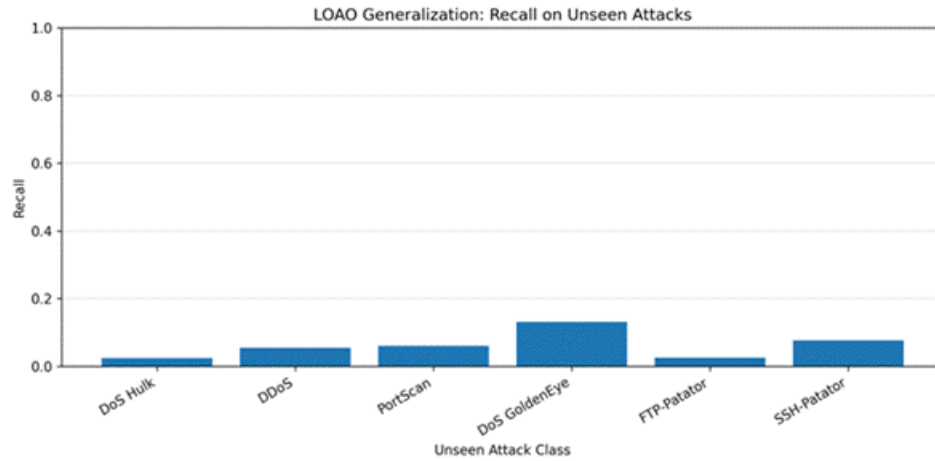


Figure 5.3 Recall of the CNN-GRU model on the held-out attack families

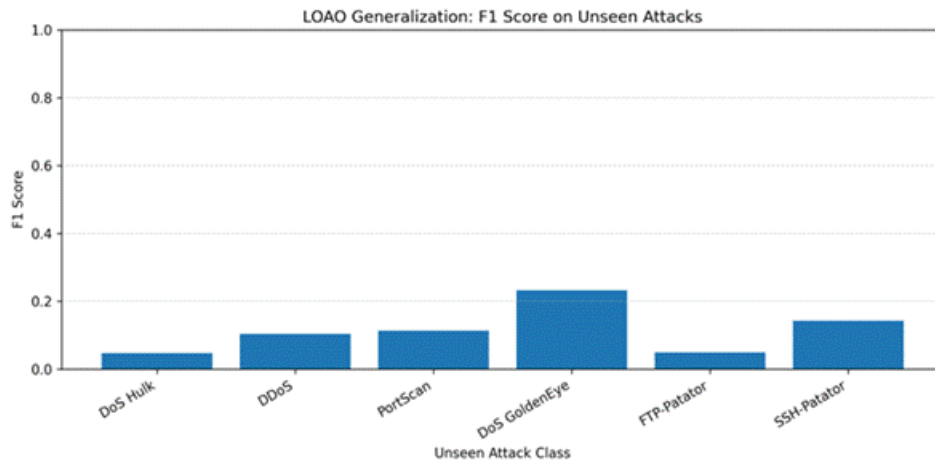


Figure 5.4 F1-score of the CNN-GRU model on the held-out attack families

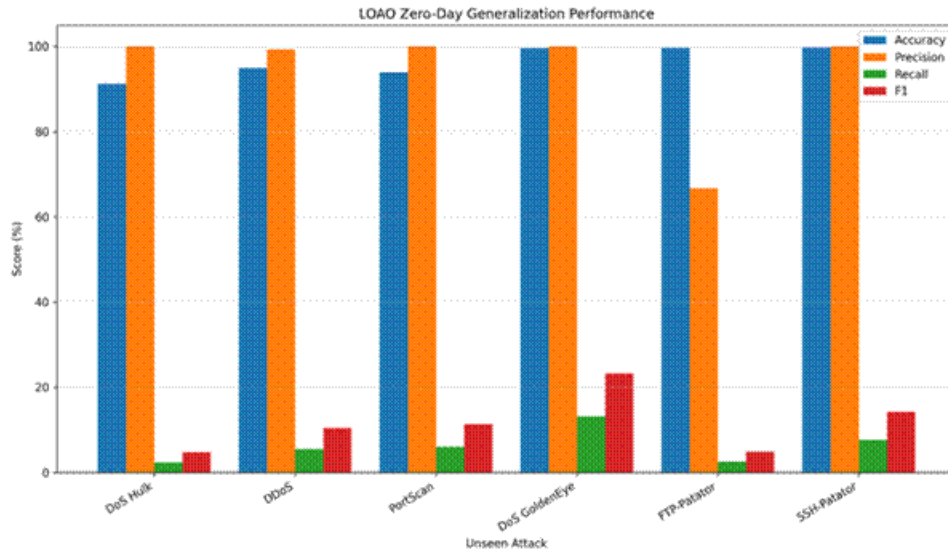


Figure 5.5 Comparative LOAO performance across the evaluated attack families

The figures show that the recall of every family remains low, and that the level differs from one family to the next. Reading the bar charts approximately, DoS GoldenEye has the highest recall and F1-score of the six, at a recall of roughly 13% and an F1-score of roughly 23%. SSH-Patator, PortScan and DDoS follow with small values, and DoS Hulk and FTP-Patator are lowest, at about 2% recall. Precision is high for most families, whereas recall is low, which suggests that the few flows the model flags are usually correct while most of the unseen attack is not flagged at all. Table 5.2 summarises this qualitative reading.

Table 5.2 Qualitative summary of LOAO recall (read from Figure 5.3)

Held-out family	Relative recall
DoS GoldenEye	Highest of the six, but still low
SSH-Patator	Low, second highest
PortScan	Low
DDoS	Low

DoS Hulk	Among the lowest
FTP-Patator	Among the lowest

5.5 FEATURE-SPACE ANALYSIS

Figure 5.6 shows the t-SNE plot of the balanced Top-30 subset. It gives a visual idea of how the traffic classes are laid out and where they lie close to each other. Some classes form compact groups of their own, while others are spread out or sit near other classes. When an attack family lies in a region that is not well covered by the classes used in training, or that overlaps benign traffic, the model has little basis for recognising it. The plot has to be treated as a qualitative diagnostic. t-SNE distorts distances and does not prove how separable the classes are in the original space.

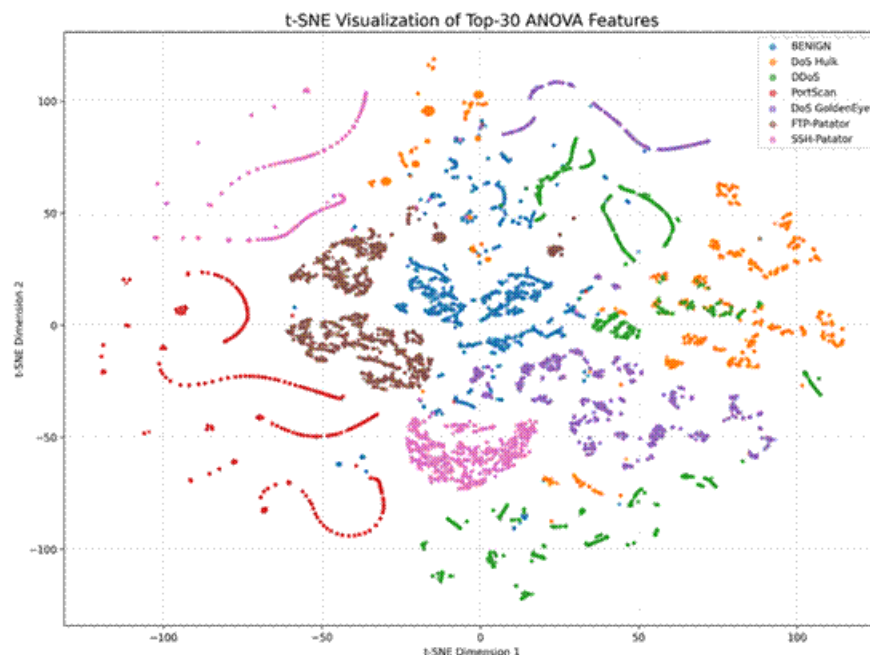


Figure 5.6 t-SNE visualisation of the Top-30 feature representation across traffic classes

5.6 DISCUSSION

Three observations follow from the experiments. First, a large gap separates closed-set classification from zero-day generalization. The same architecture that gives close to 100% on known classes recovers only a small fraction of an unseen attack family. Second, the result is not the same for every family, so an average figure hides important detail. A system might cope acceptably with one kind of unseen attack and almost completely miss another. Third, the data audit shows that the ordinary train-test protocol can flatter a model, both because of repeated rows and because the test classes are always present in training.

These points support the use of an attack-family-level protocol as a stricter test of intrusion detection models. They also indicate that accuracy on a random split should not be the only figure quoted when a system is presented as ready for real networks.

5.7 LIMITATIONS

- Only the CNN-GRU model has been evaluated so far. Comparison with Random Forest, CNN alone and GRU alone is planned but not part of this phase.
- Only six attack families and one dataset (CICIDS2017) are used, so the findings may not carry over to other datasets or to different attack types.
- The leakage audit was made on the Top-30 feature space, and the reasons for the large overlap have not been analysed in detail.
- t-SNE is qualitative, and UMAP and learned-representation analysis are not yet done.
- The LOAO results come from one training run for each family, so no measure of run-to-run variation is available.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project built an intelligent network intrusion detection system with a hybrid CNN-GRU model on CICIDS2017 and studied it beyond the usual accuracy figures. The pipeline covers data cleaning, ANOVA-based feature selection, sequence construction of 10 flows, and multi-class classification of seven traffic types. With the conventional stratified split the model achieved near-perfect accuracy and F1-score.

A closer examination changed this picture. The leakage audit found 45,848 feature rows shared between training and test data and 81 label-inconsistent duplicate signatures, so the closed-set result cannot be taken as proof of real-world strength. Under the Leave-One-Attack-Out protocol the model reached a mean recall of only 6.23% and a mean F1-score of 11.50% on six unseen attack families, and the outcome varied from one family to another. A t-SNE analysis of the Top-30 feature space provided qualitative support for interpreting these differences.

The main lesson is that deep learning is very effective at recognising attack types it has been trained on, but that zero-day detection remains an open problem, and that both attack-family-level evaluation and data validation are necessary before an intrusion detection model is judged.

6.2 FUTURE WORK

- Add Random Forest, CNN and GRU models and evaluate them with the same LOAO protocol, so the model families can be compared fairly.
- Extend the feature-space study with UMAP and with the internal representations learned by the networks, and support the visual findings with

quantitative measures.

- Identify the source of the train-test overlap and rebuild the split so that repeated rows cannot appear on both sides.
- Repeat the LOAO experiments several times and report the variation between runs.
- Investigate methods that may improve detection of unseen attacks, such as anomaly-based or open-set learning, and test them on further datasets.
- Complete the interactive demonstration into a working monitoring tool for live or replayed traffic.

REFERENCES

- [1] S. S. Bamber, A. V. R. Katkuri, S. Sharma, and M. Angurala, "A Hybrid CNN-LSTM Approach for Intelligent Cyber Intrusion Detection System," *Computers & Security*, vol. 148, Art. no. 104146, 2025.
- [2] S. Hore, J. Ghadermazi, A. Shah, and N. D. Bastian, "A Sequential Deep Learning Framework for a Robust and Resilient Network Intrusion Detection System."
- [3] M. Arcos-Argudo, R. Bojorque, and A. Torres, "A Deterministic Comparison of Classical Machine Learning and Hybrid Deep Representation Models for Intrusion Detection on NSL-KDD and CICIDS2017," *Algorithms*.
- [4] Rushendra, K. Ramli, and P. D. Purnamasari, "Stability-Aware Evaluation of a CNN-LSTM-DQN Intrusion Detection System for Zero-Day and Drifted Network Traffic," *IIUM Engineering Journal*, vol. 27, no. 2, pp. 227–256, 2026.
- [5] M. Umer, M. Tahir, M. Sardaraz, M. Sharif, H. Elmannai, and A. D. Algarni, "Network Intrusion Detection Model Using Wrapper Based Feature Selection and

Multi Head Attention Transformers," *Scientific Reports*, vol. 15, Art. no. 28718, 2025.

[6] R. Chinnasamy, M. Subramanian, S. V. E. Sathishkumar, and J. Cho, "Deep Learning-Driven Methods for Network-Based Intrusion Detection Systems: A Systematic Review," *ICT Express*, vol. 11, no. 1, pp. 181–215, 2025.

[7] U. Otokwala, A. Petrovski, and H. Kalutarage, "Optimized Common Features Selection and Deep-Autoencoder (OCFSDA) for Lightweight Intrusion Detection in Internet of Things," *International Journal of Information Security*, vol. 23, pp. 2559–2581, 2024.

[8] Y. Imrana, Y. Xiang, L. Ali, A. Noor, and K. Sarpong, "CNN-GRU-FF: A Double-Layer Feature Fusion-Based Network Intrusion Detection System Using Convolutional Neural Network and Gated Recurrent Units," *Complex & Intelligent Systems*, vol. 10, pp. 3353–3370, 2024.

[9] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "A Detailed Analysis of the CICIDS2017 Data Set," in *Proc. Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 172–188.

[10] L. J. P. van der Maaten and G. E. Hinton, "Visualizing Data Using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

